



Tests JavaScript

Jour 1 — Introduction, Karma & Jasmine

Formation Frontend Testing · Angular Ecosystem



Programme — Jour 1

01 Introduction aux tests

02 Typologie des tests

03 Particularités du front-end JS

04 Présentation des frameworks

05 Le runner Karma

06 Installation & configuration Karma

07 Atelier — Premier run

08 Tests unitaires avec Jasmine

09 Suites, specs & organisation

10 Expectations & Matchers

11 Mocking & Spies

12 Horloge simulée & Async

| 01 · Introduction aux Tests

Pourquoi tester ? Les enjeux qualité

Pourquoi tester son code ?

- **Fiabilité** : détecter les régressions avant la production
- **Confiance** : refactorer sans crainte de casser l'existant
- **Documentation vivante** : les tests décrivent le comportement attendu
- **Vélocité** : moins de bugs en prod = moins d'interruptions

« Un code sans tests est du code hérité dès le premier commit. » — Michael Feathers



Le coût du bug selon sa détection

Phase de détection	Coût relatif
Durant le développement	× 1
En code review	× 3
En QA / recette	× 10
En production	× 100

- Plus un bug est détecté **tôt**, moins il coûte cher à corriger
- Les tests automatisés = filet de sécurité permanent

La confiance comme valeur centrale

- Les tests permettent de **livrer fréquemment** en toute sérénité
- Ils sont le fondement des pratiques **CI/CD**
- Un bon coverage = liberté d'évolution de l'architecture
- Sans tests : la dette technique s'accumule silencieusement

| 02 · Typologie des Tests

De l'unitaire à l'e2e : connaître ses outils

Les grandes familles de tests

Tests Unitaires

Testent une seule unité (fonction, classe, composant) en isolation totale avec des mocks.

Tests d'Intégration

Testent la collaboration entre plusieurs modules. Les dépendances réelles sont partiellement utilisées.

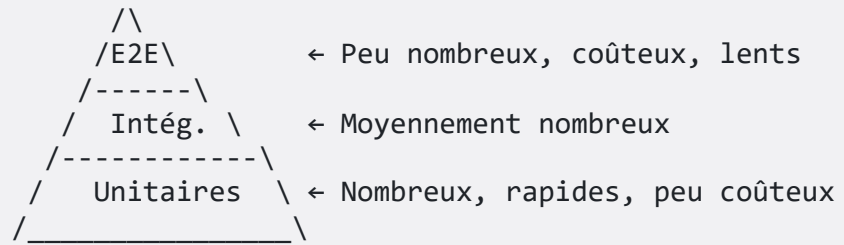
Tests E2E (End-to-End)

Simulent un utilisateur réel dans un vrai navigateur. Testent les flux complets de bout en bout.

Tests de Régression Visuelle

Compagent des captures d'écran pour détecter les changements d'UI involontaires.

▲ La Pyramide des Tests



- 70% unitaires · 20% intégration · 10% e2e
- Règle : plus le test est haut dans la pyramide, plus il est lent

⚡ Unitaires vs Intégration vs E2E

Unitaires

- Très **rapides** (ms)
- Entièrement **isolés**
- Faciles à **maintenir**
- Nombreux (centaines)
- Outillage : Jasmine, Jest, Vitest

Intégration & E2E

- Plus **lents** (secondes)
- Proches de la **réalité**
- Plus fragiles face aux changements UI
- Moins nombreux
- Outillage : Playwright, Cypress

Qu'est-ce qu'un bon test ?

- **F**ast — s'exécute rapidement
- **I**solated — indépendant des autres tests
- **R**epeatable — donne toujours le même résultat
- **S**elf-validating — passe ou échoue sans interprétation
- **T**imely — écrit au bon moment (TDD ou juste après)

Acronyme **FIRST** — les 5 qualités fondamentales d'un test

03 · Particularités du Front-End JavaScript

Tester le navigateur : défis uniques



Défis spécifiques au front-end

- Le DOM : les tests doivent simuler ou utiliser un vrai navigateur
- L'asynchronisme : Promises, Observables, setTimeout, événements
- Les dépendances globales : `window`, `document`, `localStorage`
- Le build : transpilation TypeScript, bundling, modules ES
- Les frameworks : Angular, React, Vue ajoutent leur couche d'abstraction

Environnements d'exécution

Node.js (JSDOM)

- Simule le DOM en mémoire
- Très rapide, pas de navigateur réel
- Limites : pas de CSS, pas de rendu réel
- Outillage : Jest, Vitest

Vrai Navigateur

- Rendu réel (Chrome, Firefox, Safari)
- Tests plus fidèles à la réalité
- Plus lent à démarrer
- Outillage : Karma, Web Test Runner, Playwright



L'asynchronisme : un défi majeur

```
// ❌ Test incorrect - la promesse n'est pas attendue
it('should load data', () => {
  service.getData(); // async !
  expect(component.data).toBeDefined(); // trop tôt
});

// ✅ Test correct avec async/await
it('should load data', async () => {
  await service.getData();
  expect(component.data).toBeDefined();
});
```

Angular et les tests

- Angular est conçu avec la **testabilité** en tête
- `TestBed` : utilitaire Angular pour créer un module de test
- **Injection de dépendances** : facilite le remplacement par des mocks
- `HttpClientTestingModule` : intercepte les appels HTTP
- `ComponentFixture` : wrapper pour tester les composants

| 04 · Présentation des Frameworks

Choisir les bons outils

L'écosystème de test JS

- **Jasmine** — BDD, standalone, historique Angular
 - **Jest** — populaire React/Node, JSDOM par défaut
 - **Vitest** — moderne, compatible Jest API, très rapide
 - **Mocha** — flexible, nécessite des plugins
 - **Karma** — runner navigateur (déprécié 2023)
- **Web Test Runner** — successeur de Karma (Modern Web)
 - **Playwright** — e2e multi-navigateur, très robuste
 - **Cypress** — e2e avec DX excellente
 - **Testing Library** — complément pour tester via l'UI

Positionnement des outils

Outil	Type	Environnement	Statut 2026
Karma	Runner	Navigateur réel	⚠ Déprécié
Jasmine	Framework	Karma/Node	✅ Stable (v5.9)
Jest	Framework+Runner	JSDOM/Node	✅ Très populaire
Vitest	Framework+Runner	Node/Navigateur	🚀 Angular 20 default
Playwright	E2E	Navigateur réel	🚀 Standard e2e

Karma : contexte important

- Karma a été déprécié en avril 2023 par l'équipe Karma-Runner
- Angular CLI maintient le support pour les projets existants
- Angular 20 : Vitest devient le runner par défaut pour les nouveaux projets
- Les tests Jasmine existants **restent compatibles** (seul le runner change)
- Cette formation couvre Karma pour comprendre l'existant + la migration

Notre stack pour la formation

1

Karma — Runner navigateur historique d'Angular (Jour 1)

2

Jasmine — Framework de test BDD complet (Jour 1)

3

Playwright — Tests e2e modernes multi-navigateur (Jour 2)

4

Angular TestBed — Utilitaires de test Angular intégrés

| 05 · Le Runner Karma

Architecture, fonctionnement et relation avec Node.js



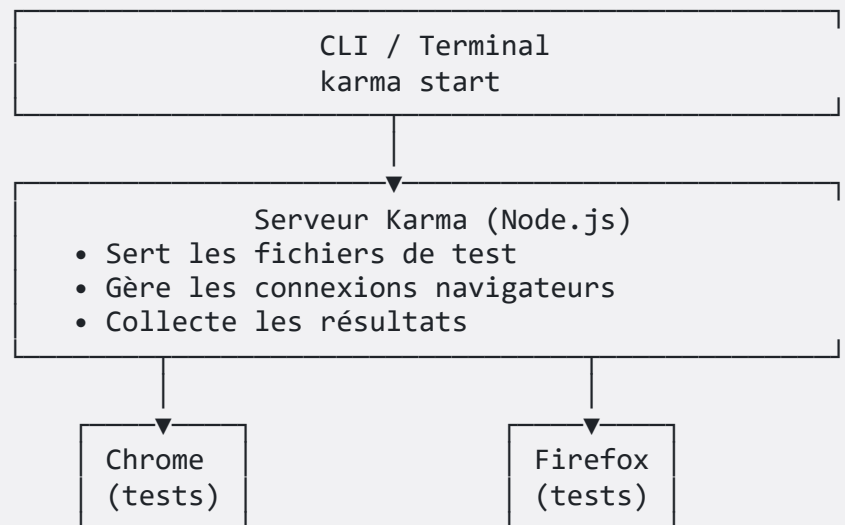
Qu'est-ce que Karma ?

- Karma est un *test runner*, pas un framework de test
- Son rôle : lancer les tests dans un ou plusieurs navigateurs
- Il ne s'occupe pas d'écrire les tests (c'est Jasmine ou Mocha)
- Créé par l'équipe Angular (ex-AngularJS), open source
- Fonctionne en tant que serveur HTTP local

Karma = le chef d'orchestre qui dit aux navigateurs d'exécuter les tests



Architecture de Karma



Relation Karma Node.js

- Karma est une application Node.js (`npm install karma`)
- Il utilise Node.js pour :
 - Démarrer le **serveur HTTP** (port 9876 par défaut)
 - Surveiller les **changements de fichiers** (watch mode)
 - **Agréger** les résultats de tous les navigateurs
 - Écrire les **rapports** (console, junit, coverage)
- Les tests eux-mêmes s'exécutent dans le **navigateur**, pas dans Node

Relation Karma Navigateurs

- Karma ouvre automatiquement les navigateurs configurés
- Chaque navigateur charge la page Karma via HTTP
- Cette page contient : le framework + vos fichiers de test + vos sources
- Les navigateurs envoient les résultats au serveur via WebSocket
- Karma peut piloter : Chrome, Firefox, Safari, Edge, IE (via plugins)



Les Plugins Karma

- `karma-jasmine` — intégration framework Jasmine
- `karma-chrome-launcher` — lancement Chrome
- `karma-firefox-launcher` — lancement Firefox
- `karma-coverage` — rapports de couverture Istanbul
- `karma-junit-reporter` — rapport XML pour CI
- `karma-spec-reporter` — affichage console amélioré
- `karma-mocha` — alternative à Jasmine
- `karma-webpack` — bundling Webpack

| 06 · Installation & Configuration Karma

Setup complet d'un projet Angular avec Karma



Installation — Projet Angular existant

```
# Angular CLI génère déjà Karma avec ng new
ng new mon-projet --routing --style=scss

# Les dépendances Karma sont incluses par défaut
# dans package.json (Angular < 20) :
# karma, karma-chrome-launcher, karma-coverage
# karma-jasmine, karma-jasmine-html-reporter

# Lancer les tests
ng test
```



Installation manuelle (hors Angular CLI)

```
# Installation globale du CLI Karma
npm install -g karma-cli

# Installation des packages nécessaires
npm install --save-dev \
  karma \
  karma-jasmine \
  karma-chrome-launcher \
  karma-coverage \
  jasmine-core

# Initialisation interactive
karma init karma.conf.js
```



karma.conf.js — Structure de base

```
module.exports = function(config) {
  config.set({
    basePath: '',
    frameworks: ['jasmine', '@angular-devkit/build-angular'],
    plugins: [
      require('karma-jasmine'),
      require('karma-chrome-launcher'),
      require('karma-jasmine-html-reporter'),
      require('karma-coverage'),
      require('@angular-devkit/build-angular/plugins/karma')
    ],
    client: {
      jasmine: { random: true, seed: '4321' },
      clearContext: false
    },
    // ... suite
  });
};
```



karma.conf.js — Navigateurs & rapports

```
// Suite de karma.conf.js
coverageReporter: {
  dir: require('path').join(__dirname, './coverage/mon-projet'),
  subdir: '.',
  reporters: [
    { type: 'html' },
    { type: 'text-summary' },
    { type: 'lcovonly' }
  ]
},
reporters: ['progress', 'kjhtml'],
browsers: ['Chrome'],
// Pour CI : headless
// browsers: ['ChromeHeadless'],
singleRun: false, // false = watch mode
restartOnFileChange: true
```




Modes d'exécution Karma

```
# Mode watch (développement) - relance à chaque sauvegarde
ng test

# Mode single run (CI)
ng test --watch=false

# Avec rapport de couverture
ng test --code-coverage

# Sur navigateur headless (CI sans interface graphique)
ng test --browsers=ChromeHeadless

# Combiné CI
ng test --watch=false --code-coverage --browsers=ChromeHeadless
```



Rapport de couverture Istanbul

```
===== Coverage summary =====  
Statements   : 85.71% ( 42/49 )  
Branches     : 75.00% ( 6/8 )  
Functions    : 88.89% ( 8/9 )  
Lines        : 85.71% ( 42/49 )  
=====
```

- **Statements** : lignes de code exécutées
- **Branches** : chemins conditionnels testés (`if/else`)
- **Functions** : fonctions appelées pendant les tests
- **Lines** : lignes physiques exécutées

angular.json — Configuration du builder test

```
"test": {  
  "builder": "@angular-devkit/build-angular:karma",  
  "options": {  
    "main": "src/test.ts",  
    "polyfills": ["zone.js", "zone.js/testing"],  
    "tsConfig": "tsconfig.spec.json",  
    "karmaConfig": "karma.conf.js",  
    "assets": ["src/favicon.ico", "src/assets"],  
    "styles": ["src/styles.scss"],  
    "codeCoverage": false,  
    "codeCoverageExclude": ["src/environments/**"]  
  }  
}
```

Structure des fichiers de test Angular

```
src/
├── app/
│   ├── components/
│   │   ├── user/
│   │   │   ├── user.component.ts
│   │   │   └── user.component.spec.ts ← colocalisé
│   ├── services/
│   │   ├── user.service.ts
│   │   └── user.service.spec.ts
│   └── app.component.spec.ts
└── test.ts ← bootstrap des tests
```

- Convention Angular : **colocalisé** avec le fichier source
- Extension **.spec.ts** obligatoire pour Karma
- **test.ts** : point d'entrée qui charge tous les ***.spec.ts**



Migration Karma → Vitest (Angular 20+)

```
# Angular 20+ : Vitest est le défaut pour ng new
ng new mon-projet # Vitest sélectionné automatiquement

# Migration d'un projet existant avec ng update
ng update @angular/cli

# Ou choisir explicitement avec le schématic
ng generate @angular/core:vitest
```

*Les tests Jasmine existants **ne changent pas** — seule la config du runner change*

Atelier 1 — Premier run Karma

 Référence : *exercices-jour1.md* — Exercice 1

Atelier 1 — Objectifs

- Créer un projet Angular avec `ng new`
- Explorer la structure des fichiers de test générés
- Lancer `ng test` et observer le navigateur Karma
- Analyser le rapport de couverture HTML
- Modifier un test et observer le rechargement à chaud

Durée estimée : 25 minutes

Référence : `exercices-jour1.md` → Exercice 1

| 07 · Tests Unitaires avec Jasmine

Le framework BDD de référence pour Angular

Jasmine — Présentation

- Framework de test **BDD** (Behavior-Driven Development)
- **Zéro dépendance** : pas de DOM, pas de bibliothèques externes requises
- Syntaxe lisible : `describe` , `it` , `expect`
- Version actuelle : **Jasmine 5.9**
- Utilisé par défaut dans Angular avec Karma et Web Test Runner
- Peut aussi tourner directement en **Node.js**

Philosophie BDD

En tant que [rôle]
Je veux [action]
Afin de [bénéfice]

Traduit en Jasmine :

```
describe('UserService', () => {  
  it('should return user by id', () => {  
    // Arrange – Given (étant donné)  
    // Act – When (quand)  
    // Assert – Then (alors)  
  });  
});
```

| 08 · Suites, Specs & Organisation

La structure d'un fichier de test Jasmine



Structure de base Jasmine

```
// Suite principale
describe('CalculatorService', () => {

  // Suite imbriquée
  describe('add()', () => {

    // Spec (test individuel)
    it('should return the sum of two numbers', () => {
      const calc = new CalculatorService();
      const result = calc.add(2, 3);
      expect(result).toBe(5);
    });

  });

});
```

Vocabulaire Jasmine

Terme	Rôle	Équivalent
<code>describe()</code>	Suite — groupe de tests	<code>@Suite</code> en JUnit
<code>it()</code>	Spec — test individuel	<code>@Test</code> en JUnit
<code>expect()</code>	Assertion — vérification	<code>assert</code> en JUnit
<code>beforeEach()</code>	Setup — avant chaque test	<code>@BeforeEach</code>
<code>afterEach()</code>	Teardown — après chaque test	<code>@AfterEach</code>
<code>beforeAll()</code>	Exécuté une seule fois avant tous	<code>@BeforeAll</code>
<code>afterAll()</code>	Exécuté une seule fois après tous	<code>@AfterAll</code>

Setup & Teardown

```
describe('UserComponent', () => {
  let component: UserComponent;
  let fixture: ComponentFixture<UserComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [UserComponent]
    }).compileComponents();

    fixture = TestBed.createComponent(UserComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  afterEach(() => {
    fixture.destroy(); // nettoyage
  });
});
```

Focus & Skip

```
// fdescribe / fit : focus – seuls ces tests tournent
fdescribe('UserService', () => {
  fit('should login', () => { /* seul ce test tourne */ });
});

// xdescribe / xit : skip – ces tests sont ignorés
xdescribe('FeatureInProgress', () => {
  xit('not yet implemented', () => { });
});
```

⚠ *Ne jamais committer du code avec `fdescribe` ou `fit` — cela masque les autres tests !*

Organisation des fichiers dans Angular

```
// user.service.spec.ts
import { TestBed } from '@angular/core/testing';
import { UserService } from '../user.service';
import { HttpClientTestingModule } from '@angular/common/http/testing';

describe('UserService', () => {
  let service: UserService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
      providers: [UserService]
    });
    service = TestBed.inject(UserService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });
});
```


⚙️ TestBed — Le cœur du test Angular

```
// Configuration minimale
TestBed.configureTestingModule({
  imports: [...],      // modules Angular nécessaires
  declarations: [...], // composants, directives, pipes
  providers: [...]     // services et leurs substituts
});

// Obtenir une instance de service
const service = TestBed.inject(UserService);

// Créer un composant
const fixture = TestBed.createComponent(MyComponent);
const component = fixture.componentInstance;
```

| 09 · Expectations & Matchers

Vérifier les résultats avec précision

✓ La structure d'une assertion

```
expect(valeurTestée).matcher(valeurAttendue)
```

```
// Exemples
```

```
expect(result).toBe(42);  
expect(user.name).toEqual('Alice');  
expect(items).toContain('apple');  
expect(fn).toThrow();  
expect(spy).toHaveBeenCalled();
```

- `expect()` crée un objet d'expectation
- Le `matcher` définit la vérification
- `not` inverse le matcher : `expect(x).not.toBe(null)`



Matchers — Égalité & Identité

```
// toBe : égalité stricte (===), référence pour les objets
expect(42).toBe(42);           // ✔
expect({a: 1}).toBe({a: 1});  // ✖ références différentes

// toEqual : égalité profonde (deep equality)
expect({a: 1}).toEqual({a: 1}); // ✔
expect([1,2,3]).toEqual([1,2,3]); // ✔

// toBeNull / toBeUndefined / toBeDefined
expect(null).toBeNull();
expect(undefined).toBeUndefined();
expect('hello').toBeDefined();
```



Matchers — Vérités & Types

```
// Truthy / Falsy
expect(1).toBeTruthy();
expect(0).toBeFalsy();
expect('').toBeFalsy();
expect('hello').toBeTruthy();

// Instances
expect(new Error()).toBeInstanceOf(Error);
expect([]).toBeInstanceOf(Array);

// NaN
expect(NaN).toBeNaN();
expect(42).not.toBeNaN();
```



Matchers — Nombres

```
// Comparaisons numériques
expect(10).toBeGreaterThan(5);
expect(3).toBeLessThan(10);
expect(10).toBeGreaterThanOrEqual(10);
expect(3).toBeLessThanOrEqual(3);

// Précision décimale
expect(0.1 + 0.2).toBeCloseTo(0.3, 5);
// 2ème argument : nombre de décimales à comparer
```



Matchers — Chaînes & Tableaux

```
// Chaînes
expect('Hello World').toContain('World');
expect('user@example.com').toMatch(/^[\w-]+@/);
expect('test string').toMatch('string');

// Tableaux
expect([1, 2, 3]).toContain(2);
expect([1, 2, 3]).toHaveLength(3);

// Objets partiels (jasmine.objectContaining)
expect({name: 'Alice', age: 30}).toEqual(
  jasmine.objectContaining({ name: 'Alice' })
);
```



Matchers — Exceptions

```
// toThrow
expect(() => {
  JSON.parse('invalid json');
}).toThrow();

// toThrowError : avec message ou type
expect(() => {
  throw new Error('Not found');
}).toThrowError('Not found');

expect(() => {
  throw new TypeError('type error');
}).toThrowError(TypeError);
```


Custom Matchers

```
// Définir un matcher personnalisé
beforeEach(() => {
  jasmine.addMatchers({
    toBeValidEmail: () => ({
      compare: (actual: string) => ({
        pass: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(actual),
        message: `Expected ${actual} to be a valid email`
      })
    })
  });
});

// Utilisation
expect('user@example.com').toBeValidEmail();
expect('not-an-email').not.toBeValidEmail();
```

Atelier 2 — Matchers Jasmine

 Référence : *exercices-jour1.md* — Exercice 2

Atelier 2 — Objectifs

- Écrire des tests pour un service `CalculatorService`
- Pratiquer les matchers `toBe`, `toEqual`, `toContain`
- Vérifier les exceptions avec `toThrow`
- Créer un custom matcher `toBePositive`
- Comprendre la différence `toBe` vs `toEqual`

Durée estimée : 30 minutes

Référence : `exercices-jour1.md` → Exercice 2

| 10 · Mocking & Spies

Isoler le code testé de ses dépendances



Pourquoi mocker ?

- Les tests unitaires testent **une seule unité** en isolation
- Les dépendances réelles peuvent être :
 - **Lentes** (appels HTTP, base de données)
 - **Aléatoires** (générateurs de nombres, dates)
 - **Indisponibles** (service externe)
 - **Difficiles à configurer** (environnement spécifique)
- Les **mocks** remplacent les vraies dépendances par des doublures



Les Spies Jasmine

```
// spyOn : espionner une méthode existante
const userService = TestBed.inject(UserService);
const spy = spyOn(userService, 'getUser');

// Vérifier qu'elle a été appelée
expect(spy).toHaveBeenCalled();
expect(spy).toHaveBeenCalledWith(42);
expect(spy).toHaveBeenCalledTimes(1);

// Inspecter les appels
const args = spy.calls.mostRecent().args;
const allCalls = spy.calls.all();
```



Spy — Contrôler les retours

```
const spy = spyOn(userService, 'getUser');

// Retourner une valeur fixe
spy.and.returnValue({ id: 1, name: 'Alice' });

// Retourner une valeur de manière séquentielle
spy.and.returnValue('first', 'second', 'third');

// Retourner une promesse
spy.and.returnValue(Promise.resolve({ id: 1 }));

// Appeler la vraie implémentation
spy.and.callThrough();

// Exécuter une fonction custom
spy.and.callFake((id: number) => ({ id, name: 'Mock' }));
```



Spy — Simuler des erreurs

```
const spy = spyOn(userService, 'getUser');

// Lancer une exception
spy.and.throwError('Service unavailable');

// Rejeter une promesse
spy.and.returnValue(Promise.reject(new Error('Not found')));

// Tester la gestion d'erreur
it('should handle service error', async () => {
  spy.and.returnValue(Promise.reject(new Error('500')));
  await expectAsync(component.loadUser()).toBeRejected();
});
```


jasmine.createSpy & createSpyObj

```
// Créer un spy standalone (sans méthode existante)
const mockFn = jasmine.createSpy('myFunction');
mockFn('arg1');
expect(mockFn).toHaveBeenCalled('arg1');

// Créer un objet entier avec des spies
const mockService = jasmine.createSpyObj('UserService',
  ['getUser', 'saveUser', 'deleteUser']
);

// Configurer les retours
mockService.getUser.and.returnValue({ id: 1, name: 'Alice' });
mockService.saveUser.and.returnValue(Promise.resolve(true));
```



Mocking dans Angular — Providers

```
beforeEach(() => {  
  const mockUserService = jasmine.createSpyObj('UserService',  
    ['getUser', 'getUsers']  
  );  
  mockUserService.getUsers.and.returnValue(of([]));  
  
  TestBed.configureTestingModule({  
    declarations: [UserListComponent],  
    providers: [  
      { provide: UserService, useValue: mockUserService }  
    ]  
  });  
});
```



Mocking HTTP avec HttpClientTestingModule

```
import { HttpClientTestingModule, HttpTestingController }  
  from '@angular/common/http/testing';  
  
describe('UserService', () => {  
  let httpMock: HttpTestingController;  
  
  beforeEach(() => {  
    TestBed.configureTestingModule({  
      imports: [HttpClientTestingModule]  
    });  
    httpMock = TestBed.inject(HttpTestingController);  
  });  
  
  afterEach(() => httpMock.verify()); // vérifie pas de requêtes non traitées  
});
```



Mocking HTTP — Test complet

```
it('should fetch users from API', () => {  
  const mockUsers = [{ id: 1, name: 'Alice' }];  
  
  service.getUsers().subscribe(users => {  
    expect(users).toEqual(mockUsers);  
  });  
  
  // Intercepter la requête HTTP et simuler la réponse  
  const req = httpMock.expectOne('/api/users');  
  expect(req.request.method).toBe('GET');  
  req.flush(mockUsers); // simuler la réponse du serveur  
});
```

Atelier 3 — Spies & Mocking

 *Référence : exercices-jour1.md — Exercice 3*

Atelier 3 — Objectifs

- Tester un composant avec un service mocké
- Utiliser `jasmine.createSpyObj` pour simuler `UserService`
- Vérifier les appels avec `toHaveBeenCalled`
- Mocker les appels HTTP avec `HttpClientTestingModule`
- Tester les cas d'erreur

Durée estimée : 35 minutes

Référence : `exercices-jour1.md` → Exercice 3

11 · Horloge Simulée & Tests de l'Asynchronisme

Maîtriser le temps et les Promises dans les tests



Le problème du temps réel

```
// ✗ Ce test prend 5 secondes !  
it('should show notification after 5s', (done) => {  
  component.showNotification();  
  setTimeout(() => {  
    expect(component.visible).toBeTrue();  
    done();  
  }, 5000);  
});
```

- Les `setTimeout`, `setInterval` ralentissent les tests
- Les animations sont impossibles à tester normalement
- Solution : horloge simulée qui contrôle le temps



jasmine.clock() — Horloge simulée

```
describe('NotificationComponent', () => {
  beforeEach(() => jasmine.clock().install());
  afterEach(() => jasmine.clock().uninstall());

  it('should show notification after 5s', () => {
    component.showNotification(); // lance un setTimeout(fn, 5000)

    // Avancer le temps de 5 secondes instantanément
    jasmine.clock().tick(5000);

    expect(component.visible).toBeTrue();
  });
});
```



Contrôler Date.now()

```
beforeEach(() => {
  jasmine.clock().install();
  jasmine.clock().mockDate(new Date('2025-01-15'));
});

it('should display correct date', () => {
  const formatted = dateService.formatToday();
  expect(formatted).toBe('15/01/2025');
});

afterEach(() => jasmine.clock().uninstall());
```



Tests Asynchrones — done callback

```
it('should load data (callback style)', (done) => {  
  service.loadData().subscribe({  
    next: (data) => {  
      expect(data).toBeDefined();  
      done(); // signale que le test est terminé  
    },  
    error: done.fail  
  });  
});
```

- Style **legacy** — encore utilisé avec les Observables
- `done.fail(error)` signale un échec asynchrone
- Timeout par défaut : 5 secondes



Tests Asynchrones — async/await

```
// Avec les Promises
it('should fetch user', async () => {
  const user = await service.getUserById(1);
  expect(user.name).toBe('Alice');
});

// expectAsync - matchers asynchrones
it('should reject on 404', async () => {
  const promise = service.getUserById(999);
  await expectAsync(promise).toBeRejectedWithError('Not found');
});
```



Tests Asynchrones — fakeAsync

```
import { fakeAsync, tick } from '@angular/core/testing';

it('should complete after delay', fakeAsync(() => {
  let completed = false;

  setTimeout(() => completed = true, 1000);

  tick(1000); // avance le temps de 1000ms

  expect(completed).toBeTrue();
}));
```

`fakeAsync` + `tick` : la combinaison Angular recommandée



fakeAsync — Avec les Observables

```
import { fakeAsync, tick, flush } from '@angular/core/testing';

it('should load users after debounce', fakeAsync(() => {
  component.searchControl.setValue('alice');
  tick(300); // passer le debounceTime(300)
  fixture.detectChanges();

  expect(component.users.length).toBeGreaterThan(0);
}));

// flush() : exécute toutes les macrotâches en attente
it('should complete all async tasks', fakeAsync(() => {
  component.init();
  flush();
  expect(component.loaded).toBeTrue();
}));
```



Simuler les appels Ajax

```
it('should display users from API', fakeAsync(() => {  
  const mockUsers = [{ id: 1, name: 'Alice' }];  
  
  // L'appel HTTP est intercepté par HttpTestingController  
  component.loadUsers();  
  
  const req = httpMock.expectOne('/api/users');  
  req.flush(mockUsers); // simuler la réponse  
  
  tick(); // traiter les microtâches  
  fixture.detectChanges();  
  
  expect(component.users).toEqual(mockUsers);  
}));
```



Simuler les erreurs Ajax

```
it('should show error on API failure', fakeAsync(() => {  
  component.loadUsers();  
  
  const req = httpMock.expectOne('/api/users');  
  
  // Simuler une erreur réseau  
  req.error(new ProgressEvent('error'), {  
    status: 500,  
    statusText: 'Internal Server Error'  
  });  
  
  tick();  
  fixture.detectChanges();  
  
  expect(component.errorMessage).toContain('500');  
}));
```


Atelier 4 — Async & Horloge

 Référence : *exercices-jour1.md* — Exercice 4

Atelier 4 — Objectifs

- Tester un composant avec `jasmine.clock()`
- Utiliser `fakeAsync` + `tick` pour les Observables
- Tester des requêtes HTTP avec `HttpClientTestingModule`
- Simuler des erreurs réseau et vérifier la gestion d'erreur
- Utiliser `expectAsync` pour les Promises

Durée estimée : 40 minutes

Référence : `exercices-jour1.md` → Exercice 4

⚙️ Test d'un Composant Angular — Récap

```
describe('UserListComponent', () => {
  let component: UserListComponent;
  let fixture: ComponentFixture<UserListComponent>;
  let mockService: jasmine.SpyObj<UserService>;

  beforeEach(async () => {
    mockService = jasmine.createSpyObj('UserService', ['getUsers']);
    mockService.getUsers.and.returnValue(of([{ id: 1, name: 'Alice' }]));

    await TestBed.configureTestingModule({
      declarations: [UserListComponent],
      providers: [{ provide: UserService, useValue: mockService }]
    }).compileComponents();

    fixture = TestBed.createComponent(UserListComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });
});
```

Test des interactions DOM

```
it('should display user names in the list', () => {
  const items = fixture.debugElement.queryAll(
    By.css('.user-item')
  );
  expect(items.length).toBe(1);
  expect(items[0].nativeElement.textContent).toContain('Alice');
});

it('should call delete when button clicked', () => {
  const btn = fixture.debugElement.query(By.css('.delete-btn'));
  btn.triggerEventHandler('click', null);
  expect(mockService.deleteUser).toHaveBeenCalled(1);
});
```

⚙️ Test des Input & Output

```
// Tester un @Input()
it('should display the passed title', () => {
  component.title = 'My Users';
  fixture.detectChanges();
  const h1 = fixture.nativeElement.querySelector('h1');
  expect(h1.textContent).toBe('My Users');
});

// Tester un @Output()
it('should emit userSelected event on click', () => {
  let emittedUser: User | undefined;
  component.userSelected.subscribe(u => emittedUser = u);

  component.selectUser({ id: 1, name: 'Alice' });
  expect(emittedUser?.name).toBe('Alice');
});
```



Récapitulatif Jour 1

- ☒ Pyramide des tests et stratégie
 - ☒ Particularités du front-end JS
 - ☒ Karma : architecture et configuration
 - ☒ angular.json et karma.conf.js
 - ☒ Jasmine : describe / it / expect
 - ☒ beforeEach / afterEach / beforeAll
- ☒ Matchers : toBe, toEqual, toContain...
 - ☒ Custom matchers
 - ☒ Spies : spyOn, createSpyObj
 - ☒ Mocking HTTP avec HttpClientTestingModule
 - ☒ fakeAsync / tick / flush
 - ☒ jasmine.clock()

Demain — Jour 2

- Tests E2E avec Playwright (v1.48+)
- Installation et configuration (`playwright.config.ts`)
- Locators modernes : `getByRole` , `getByLabel` ...
- Page Object Model avec TypeScript
- UI Mode et Trace Viewer pour débbugger
- Tests sur Angular apps avec Playwright
- Intégration CI/CD



Bonne fin de Jour 1 !

Des questions ? Rendez-vous demain pour le Jour 2